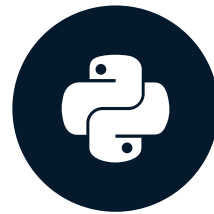


Control flow and loops

PYTHON FOR R USERS



Daniel Chen
Instructor

Whitespace matters

Whitespace and indentation is a fundamental part of writing python code and is not optional

Control flow

R

```
if (5 == 5) {  
  print('TRUE')  
}
```

"TRUE"

Python

```
if 5 == 5:  
    print('True')
```

True

if-elif-else

R

```
val <- 2

if (val == 1) {
  print('snap')
} else if (val == 2) {
  print('crackle')
} else {
  print('pop')
}
```

"crackle"

Python

```
val = 2

if val == 1:
    print('snap')
elif val == 2:
    print('crackle')
else:
    print('pop')
```

crackle

Loops

R

```
num_val <- c(1, 2, 3, 4)

for (value in seq_along(num_val)){
  print(value)
}
```

Python

```
num_val = [1, 2, 3, 4]

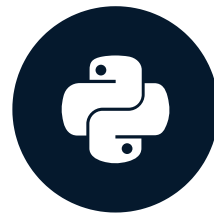
for value in num_val:
  print(value)
```

Let's practice!

PYTHON FOR R USERS

Functions

PYTHON FOR R USERS



Daniel Chen
Instructor

Simple function

R

```
my_mean <- function(x, y) {  
  num = x + y  
  dem = 2  
  num / dem  
}  
my_mean(10, 20)
```

Python

```
def my_mean(x, y):  
    num = x + y  
    dem = 2  
    return num / dem  
my_mean(10, 20)
```


R

```
my_sq <- function(x) {  
  x^2 }  
my_sq_mean <- function(y, z) {  
  (my_sq(y) + my_sq(z)) / 2 }  
my_sq_mean(10, 12)
```

122

Python

```
def my_sq(x):  
    return x ** 2  
def my_sq_mean(y, z):  
    return (my_sq(y) + my_sq(z)) / 2  
my_sq_mean(10, 12)
```

122.0

Lambda functions

R

```
add_1 <- function(x) x + 1  
function(x) x + 1
```

Python

```
def add_1(x):  
    return x + 1  
  
a1_lam = lambda x: x + 1  
a1_lam(3)
```

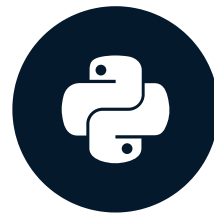
4

Let's practice!

PYTHON FOR R USERS

Comprehensions

PYTHON FOR R USERS



Daniel Chen
Instructor

Comprehensions are loops

- iterate (loop) through a list
- perform some function
- append results into a new list

List comprehension

Loop

```
data = [1, 2, 3, 4, 5]
new = []
for x in data:
    new.append(x**2)
print(new)
```

```
[1, 4, 9, 16, 25]
```

Comprehension

```
data = [1, 2, 3, 4, 5]

new = [x**2 for x in data]
print(new)
```

```
[1, 4, 9, 16, 25]
```

Dictionary comprehension

Loop

```
data = [1, 2, 3, 4, 5]
new = {}

for x in data:
    new[x] = x**2
print(new)
```

```
{1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

Comprehension

```
data = [1, 2, 3, 4, 5]

new = {x: x**2 for x in data}
print(new)
```

```
{1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```


Alternatives to `for` loop

- `for` loop to iterate
- R: `sapply` , `lapply` , `apply` functions
- Python: `map` function and `apply` method

Map

For Loop

```
def sq(x):  
    return x**2  
  
l = [1, 2, 3]  
for i in l:  
    print(sq(i))
```

```
1  
4  
9
```

Map

```
map(sq, l)
```

```
<map at 0x7f41cca38358>
```

```
list(map(sq, l))
```

```
[1, 4, 9]
```

Let's practice!

PYTHON FOR R USERS